Custom 32Bits CPU and Computer System

Design, implementation, debugging, and animated graphical demo

This document presents a complete custom CPU project built from basic digital logic up to a working computer system with memory-mapped graphics. The emphasis is on clarity, educational value, and reproducibility: each architectural decision is simple enough to study, yet rich enough to produce a visually engaging demo.

The final system includes a ROM-based control unit, a register file, an ALU with arithmetic and logic instructions, word-addressed RAM, a framebuffer connected to an LED matrix, and a program that renders an animated scene with a walker and a flying object.

Document structure

- Sections 1 to 9 describe the final architecture and implementation.
- Appendix A gives an updated step-by-step build guide aligned with the final system.
- Appendix B summarizes the demo program structure and points to the separate annotated assembly listing.

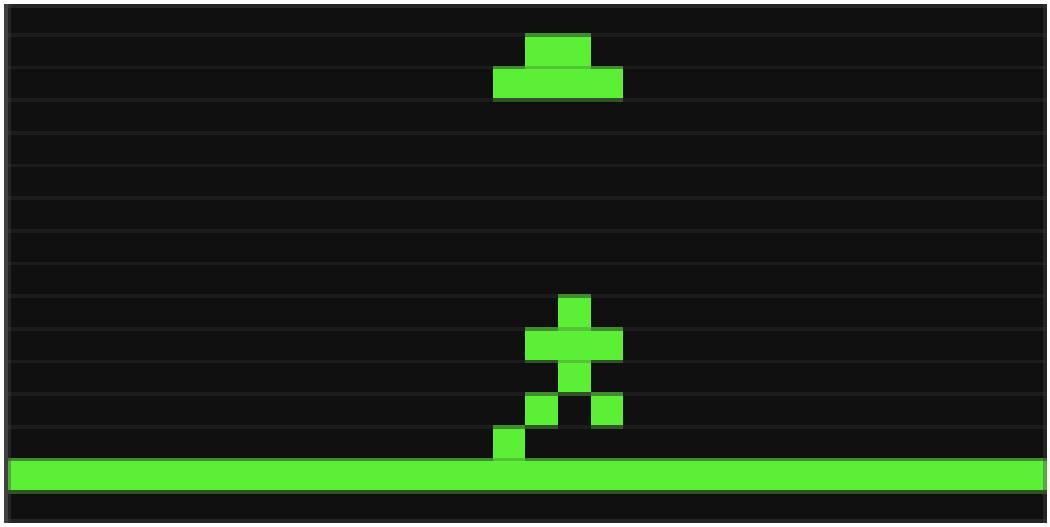


Figure 1. Final animated scene running on the custom computer system.

1. Motivation

The main goal of this project is to show how a computer can be understood as a layered system, starting from simple logic structures and ending with visible, interactive behavior. Rather than stopping at an ALU or an instruction decoder, the project continues until the machine can execute a meaningful graphical program.

Three design priorities guided the work: simplicity, observability, and completeness. Simplicity kept each component understandable. Observability made debugging possible by probing real signals. Completeness ensured that the result was not only a CPU core, but a small computer system capable of running a polished visual demo.

2. System Architecture

The system follows a simplified Harvard-style organization. Instruction memory and data memory are separate, which keeps program storage independent from runtime data and simplifies the top-level structure.

```
Instruction ROM -> CPU Core -> Data Memory / Framebuffer -> LED Matrix
```

Main components

- Instruction Memory stores 32-bit instructions in ROM and is addressed by the Program Counter.
- CPU Core contains the Program Counter, Control Unit, Register File, ALU, and routing multiplexers.
- Data Memory stores runtime data and animation state.
- Framebuffer is memory-mapped so that ordinary STORE instructions can update the display directly.

Addressing model

The final system uses word addressing. Each address refers to one 32-bit word. This is an intentional design decision: it simplifies PC handling, memory access, and branch computation. In the final implementation, the Program Counter advances by 1 instruction at a time instead of by 4 bytes.

3. Instruction Set Architecture

The CPU uses a fixed 32-bit instruction width. This keeps decoding simple and makes the architecture suitable for educational use and manual inspection in Logisim.

3.1 Instruction formats

```
R-type: [ opcode (6) ][ RD (5) ][ RS1 (5) ][ RS2 (5) ][ unused (11) ]
I-type: [ opcode (6) ][ RD (5) ][ RS  (5) ][ immediate (16) ]
J-type: [ opcode (6) ][ address (26) ]
```

R-type is used for register-register ALU operations. I-type is used for immediate arithmetic, memory access, and branches. J-type is used for unconditional jump instructions.

3.2 Register model

- General-purpose registers: R0 to R7
- Register width: 32 bits
- The design does not rely on a hardwired zero register; all registers are general-purpose in the implementation.

3.3 Final opcode table

Opcode	Mnemonic	Type	Semantics
0x01	ADDI	I	RD = RS + imm16
0x02	JMP	J	PC = target
0x04	SHL	R	RD = RS << 1
0x05	BNE	I	if RD != RS then PC = PC + 1 + imm16
0x08	ADD	R	RD = RS1 + RS2
0x09	SUB	R	RD = RS2 - RS1 (implementation-defined operand order)
0x0A	AND	R	RD = RS1 & RS2
0x0B	OR	R	RD = RS1 | RS2

0x0C	XOR	R	RD = RS1 ^ RS2
0x0D	NOT	R	RD = ~RS
0x0E	SHR	R	RD = RS >> 1
0x10	LOAD	I	RD = MEM[RS + imm16]
0x11	STORE	I	MEM[RS + imm16] = RD

3.4 Encoding example

Example instruction: ADD R1, R2, R3

opcode = ADD

RD = R1

RS1 = R2

RS2 = R3

Binary layout:

```
[ opcode ][ 00001 ][ 00010 ][ 00011 ][ unused ]
```

31-26	25-21	20-16	15-11	10-0
opcode	RD	RS1	RS2	unused
000010	00001	00010	00011	00000000000

Using fixed-width fields makes hardware decoding straightforward: the Control Unit only needs the opcode, while the register indices are routed directly to the Register File.

4. CPU Core Design

4.1 Program Counter

The Program Counter is a 32-bit register. In the final word-addressed architecture, the normal next value is:

```
PC_next = PC + 1
```

Branch instructions use the same addressing model. The final branch computation is:

```
branch_target = PC + 1 + imm16_sign_extended
```

This was an important stabilization point in the project. Using PC + 1 aligns the branch offset with the next sequential instruction, which makes manual assembly and debugging much clearer.

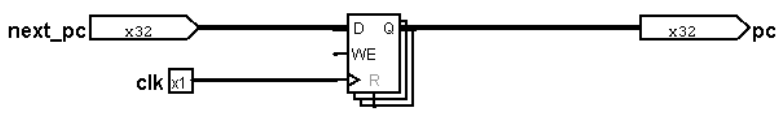


Figure 2. Program Counter implementation.

4.2 Control Unit

The Control Unit is implemented as a ROM-based lookup table. Each opcode maps directly to a control word. This approach avoids complicated combinational decode logic and makes the machine easy to inspect and modify.

- Control signals include register write enable, memory access signals, ALU source selection, write-back selection, branch, and jump.
- New instructions can be added by defining an ALU behavior and assigning a new control ROM entry.

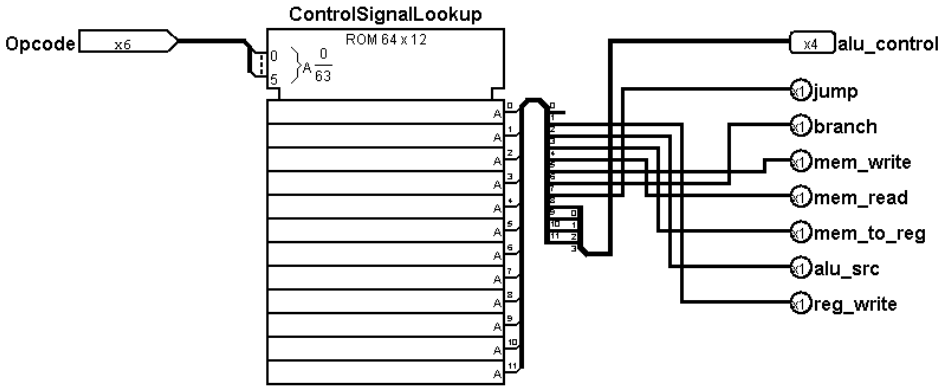


Figure 3. Control Unit based on ROM lookup.

4.3 Register File

The Register File contains eight 32-bit registers with two read ports and one write port. A decoder selects the destination register for writes, while two multiplexers select the registers presented on the read ports.

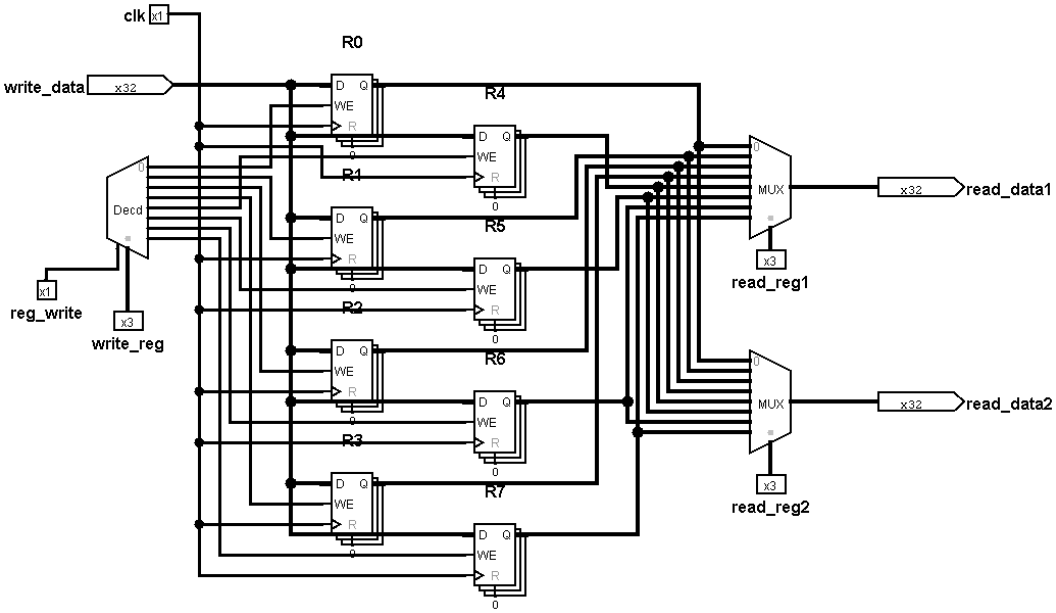


Figure 4. Register File subcircuit.

4.4 ALU

The ALU performs arithmetic, logical, and shift operations. It is implemented as parallel operation blocks feeding a result multiplexer. This structure made it easy to extend the ALU with XOR and SHR late in the project.

- Arithmetic: ADD, SUB
- Logic: AND, OR, XOR, NOT
- Shift: SHL, SHR

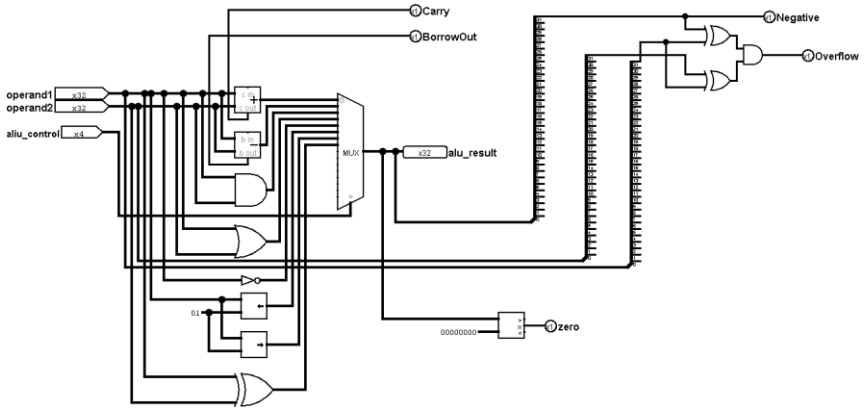


Figure 5. ALU implementation with arithmetic, logic, and shift units.

4.5 Important implementation notes

- SUB is implemented with a reversed operand order compared to textbook notation. In the final machine, the confirmed behavior is RS2 - RS1.
- Unary operations such as NOT and SHR use a single source register and were validated experimentally against ALU probes.
- The final opcode-to-operation mapping was not assumed from theory; it was verified instruction by instruction using direct alu_result checks.

5. Datapath and Data Routing

The datapath connects the Register File, ALU, memory system, and Program Counter update logic. The most important routing decisions are controlled by multiplexers.

```
Register File -> ALU -> (ALU result or memory data) -> Write-back
                 -> Data Memory / Framebuffer
```

Key multiplexers

- ALU source multiplexer: selects register data or immediate value.
- Write-back multiplexer: selects ALU result or memory data.
- PC source selection: chooses sequential execution, branch target, or jump target.

A major debugging lesson from the project was that routing logic must remain simple. Over-gating intermediate data with control signals can create misleading behavior and makes debugging much harder. The final system uses control signals primarily for selection, not for injecting zeros into otherwise valid datapaths.

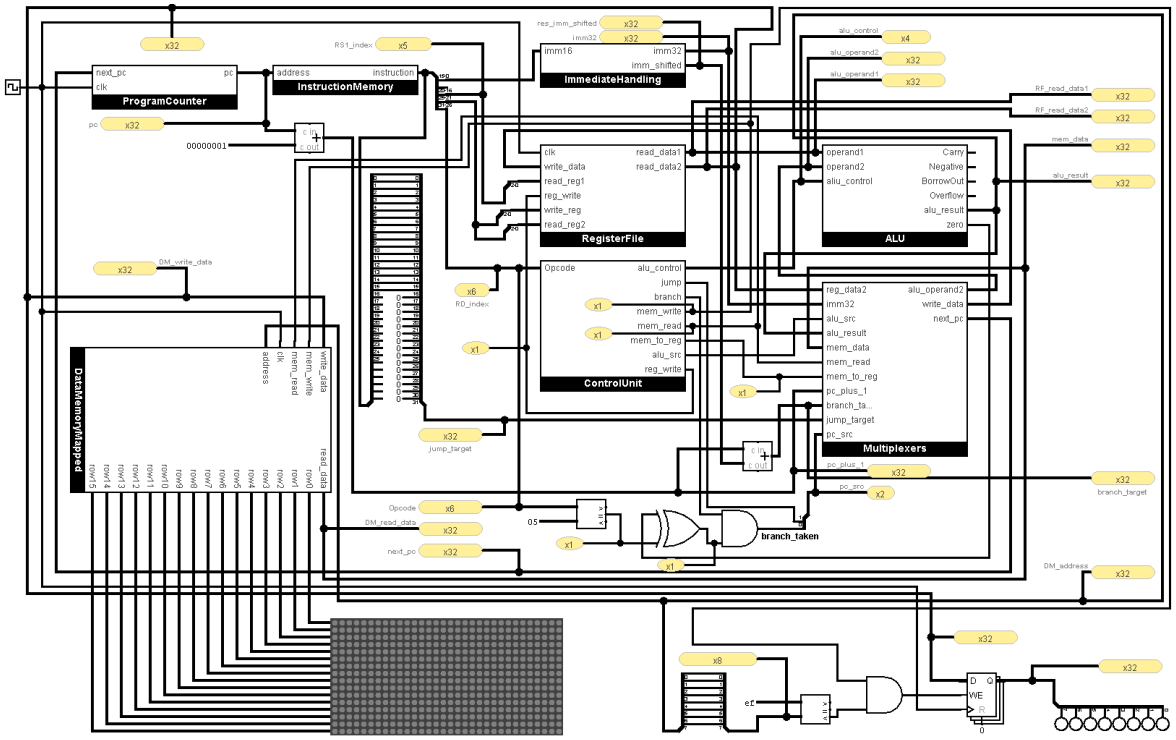


Figure 6. Top-level CPU datapath.

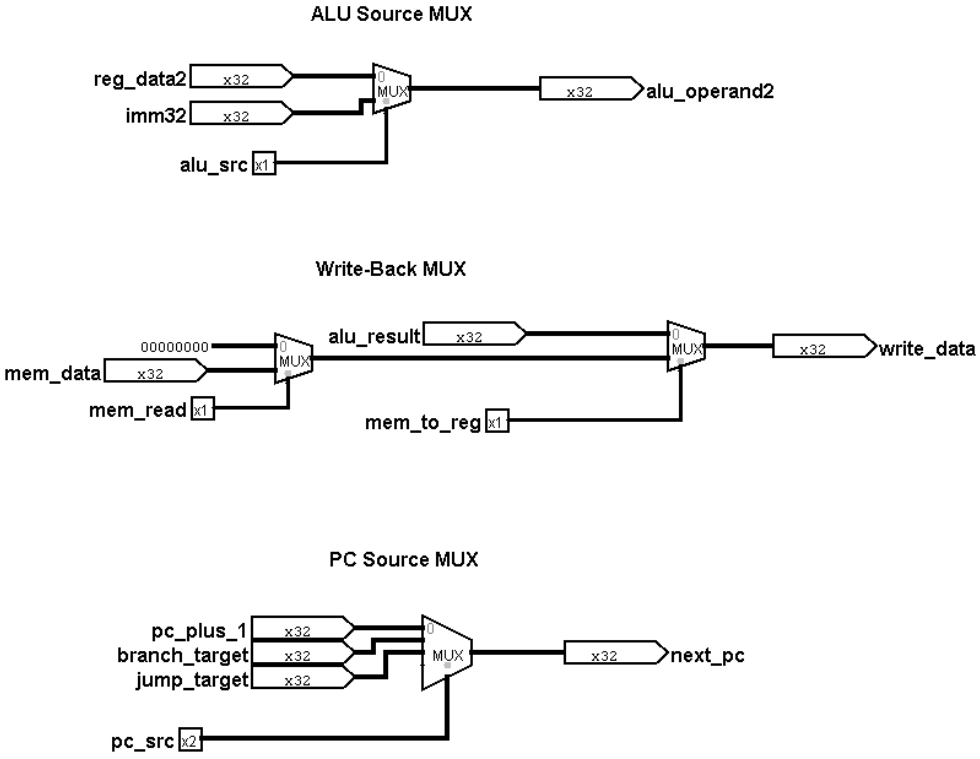


Figure 7. Multiplexers used for operand, write-back, and PC routing.

6. Memory System

6.1 Data memory

The data memory is a word-addressed RAM. It stores both ordinary runtime data and animation frame data used by the demo program. The design eventually favored runtime RAM initialization over external preload dependence, because runtime writes and reads proved to be more reliable and easier to reason about in the simulation environment.

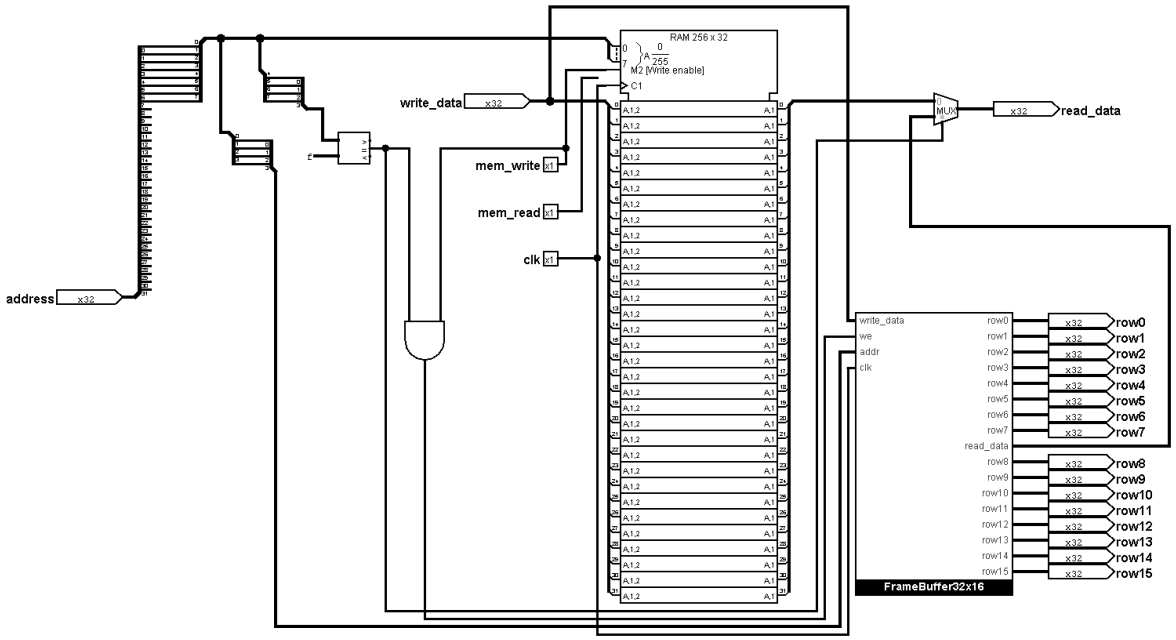


Figure 8. Data memory and memory-mapped framebuffer integration.

6.2 Memory-mapped framebuffer

A dedicated address window maps directly to the LED matrix. The final design uses addresses 0xF0 to 0xFF as display row addresses. Writing a 32-bit word to one of these addresses updates one row of the screen.

```
0xF0 - 0xFF -> display rows
```

This is one of the most powerful design choices in the project: once the framebuffer is memory-mapped, graphical output becomes a pure software problem. Any program that can generate row values can render visible graphics without any special display instructions.

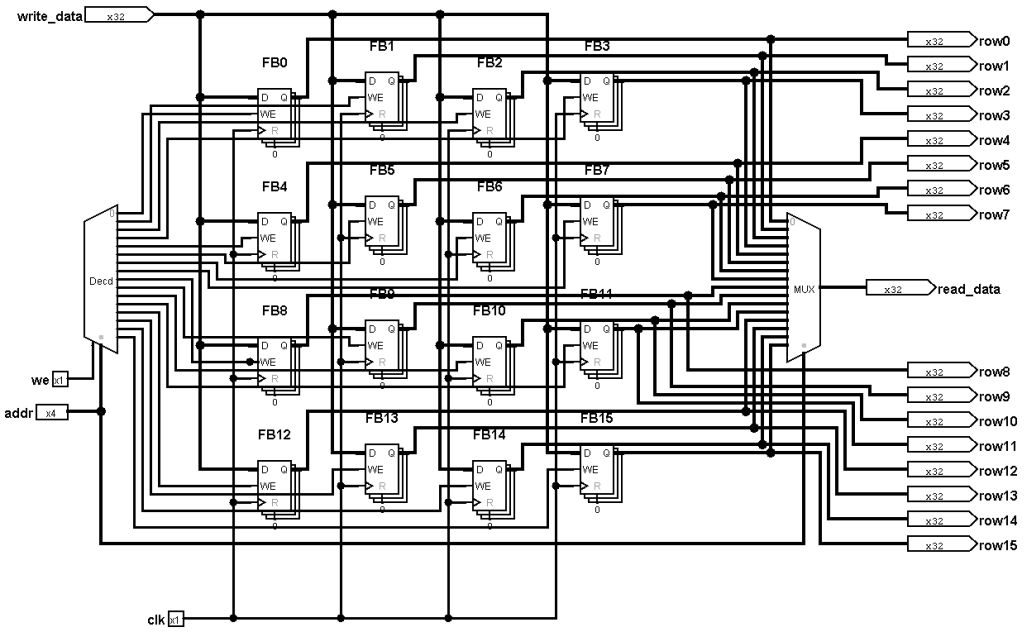


Figure 9. 32x16 framebuffer and display mapping.

7. Graphics and Demo Design

7.1 Rendering model

The framebuffer uses a row-oriented representation: one 32-bit word corresponds to one visible row. Each bit controls one pixel. This makes sprites naturally easy to represent as row lists and allows direct use of bitwise operations and shifts.

7.2 Final demo

The final demo evolved from a single moving bar into a full animated scene. The retained version balances speed, readability, and visual appeal. It shows two objects: a walking figure near the ground and a flying object in the upper part of the screen. The scene alternates its scrolling direction on restart, which creates a more dynamic and polished presentation.

- Walker: small sprite with visible pose variation
- Flying object: compact upper-screen sprite with readable motion
- Ground line: stabilizes the composition visually
- Alternating directions: increases visual interest without requiring more hardware

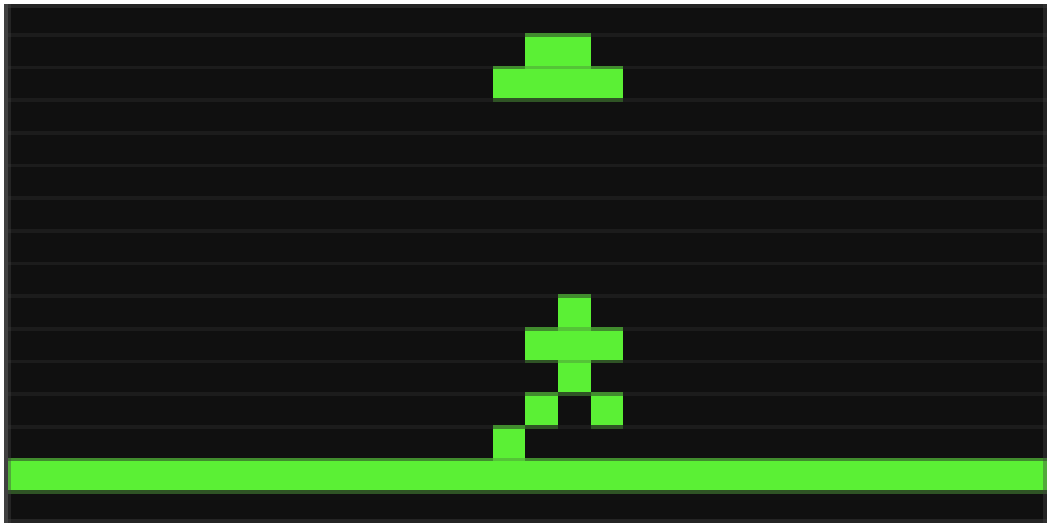


Figure 10. Example scene output on the LED matrix.

8. Performance Considerations

Animation speed is limited by instruction throughput. Every additional frame, pose, or sprite row increases the number of executed instructions per visible step. For that reason, the final demo was not chosen by artistic preference alone, but by balancing four competing constraints:

- sprite size
- pose count
- perceived smoothness
- execution speed

A larger single sprite with many poses looked good but became too slow. A scene with two smaller objects produced a better overall impression because it reduced per-frame cost while increasing visual richness.

9. Debugging and Validation

The project was validated incrementally. Instead of only testing the full CPU at the end, each component and signal path was checked in isolation. This method was essential: several problems would have been almost impossible to diagnose if only the final visual output had been observed.

Most important debugging milestones

- Verifying control ROM entries against real control outputs, not assumptions
- Confirming ALU operations by probing alu_result directly
- Correcting branch target computation from PC-relative logic
- Diagnosing memory read path behavior by comparing raw RAM output to downstream signals
- Separating tool-specific RAM preload behavior from real datapath logic
- Testing opcodes experimentally instead of inferring them from visual patterns alone

One of the strongest lessons from the project is that theoretical correctness is not enough. A design becomes trustworthy only after signals are probed under real execution and the observed behavior is reconciled with the intended architecture.

Appendix A - Updated Step-by-Step Build Guide

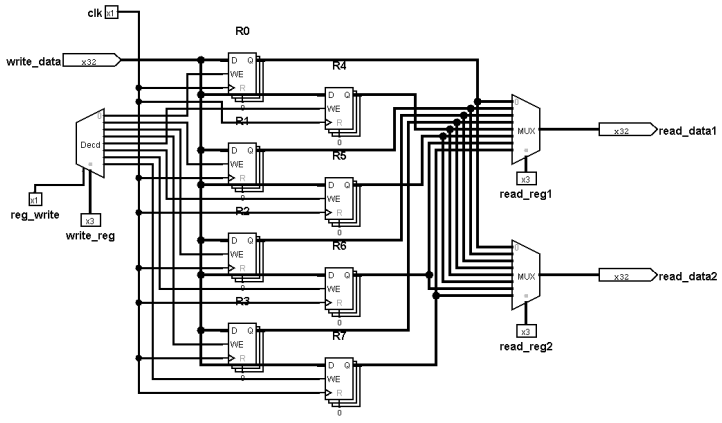
A.1 Philosophy

The system is best built bottom-up. Each stage should be completed and tested before moving to the next. This approach keeps debugging local and prevents multiple unknowns from appearing at the same time.

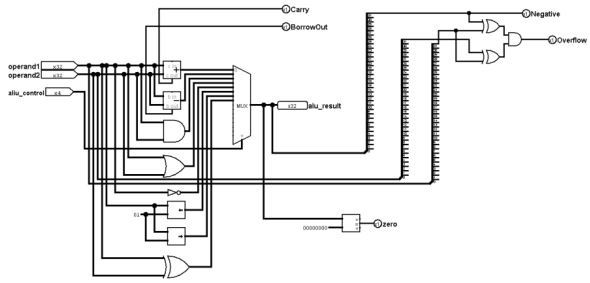
- Build one component at a time.
- Probe internal signals frequently.
- Do not add new instructions before the baseline datapath is stable.

A.2 Build sequence

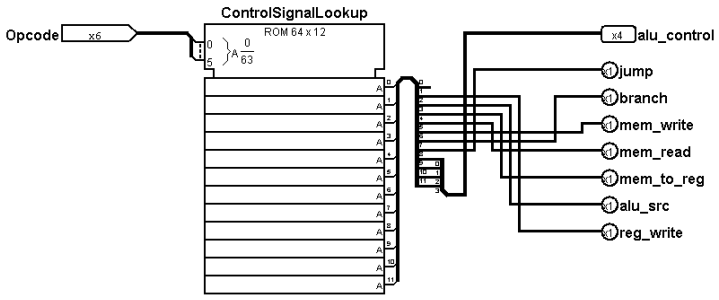
- Register File
- ALU
- Control Unit
- Program Counter
- Instruction Memory
- Data Memory
- Immediate handling
- Top-level datapath integration
- Framebuffer mapping
- First test programs
- Animated demo



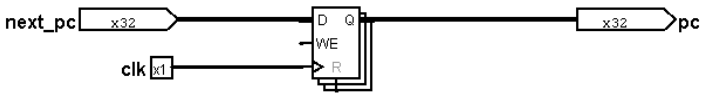
A2 visual: Register File



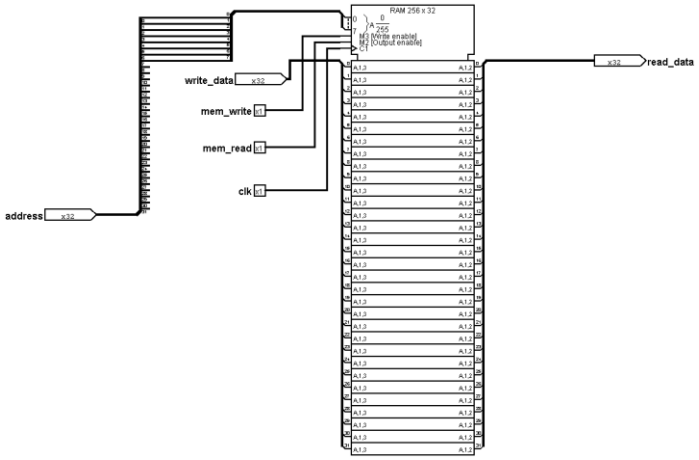
A2 visual: ALU



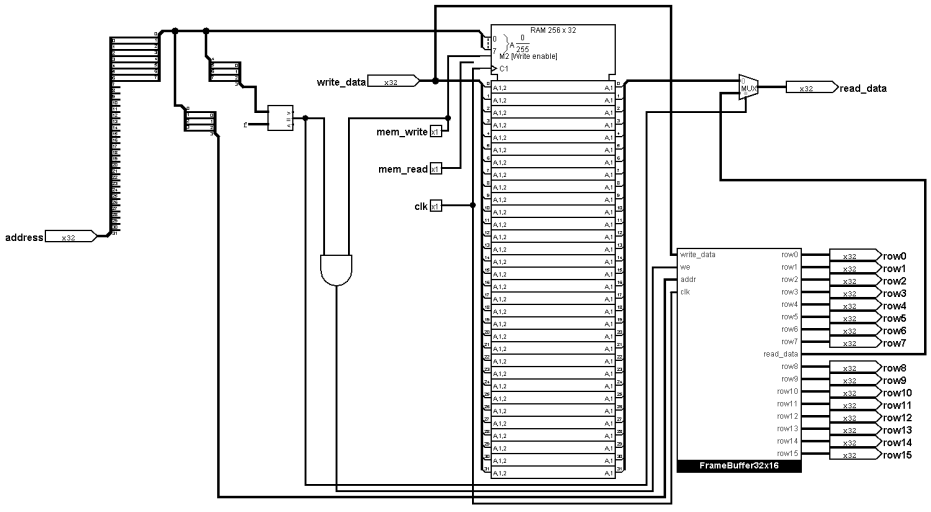
A2 visual: Control Unit



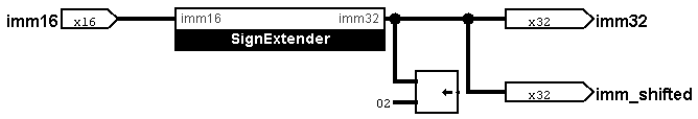
A2 visual: Program Counter



A2 visual: Instruction Memory



A2 visual: Instruction Memory with FrameBuffer support



A2 visual: Immediate Handling

A.3 Key design decisions

- Word-addressed architecture instead of byte-addressed memory
- PC + 1 + imm branch computation
- ROM-based control logic for transparency and easy modification
- Memory-mapped framebuffer for software-driven graphics
- Runtime-built animation frames instead of relying on external RAM preload

A.4 Testing strategy

Recommended validation sequence:

- Test ALU operations with direct alu_result probes.
- Test Register File reads and writes independently.
- Test STORE to framebuffer with simple fixed patterns.
- Test LOAD and STORE through RAM with minimal programs.
- Test BNE and JMP using observable branch targets.
- Only then build the full animated scene.

Appendix B - Demo Program Summary

The complete commented assembly program is provided separately. The final version uses runtime-generated frame data, memory-mapped graphics, pose variation, and alternating scene directions. Keeping the full assembly listing in a separate file improves readability of this document while still making the project reproducible.

Typical high-level structure of the demo:

- Initialize ground line
- Build pose data in RAM
- Draw scene to framebuffer
- Advance scene state
- Alternate direction on restart